38 MONGODB RELATIONSHIPS — EMBEDDING vs REFERENCING, ObjectId, populate()

theory/37 mapped Mongoose calls to SQL, including a one-line mention of populate() as Mongo's answer to JOIN. This file covers that properly: Mongo has no foreign keys or JOINs, so "relationships" between documents are a modeling choice you make, not a schema feature the database enforces for you (theory/37's section 10, CONSTRAINTS, has no Mongo equivalent — that enforcement gap is the whole reason this choice matters).

THE TWO STRATEGIES

1. EMBEDDING — store the related data INSIDE the parent document.
2. REFERENCING — store just an _id pointing at a document in another collection, and fetch it separately (or via populate()).

There's no single "correct" one — it's a tradeoff, decided per relationship, not per project.

1. EMBEDDING

```
@Schema()
class Address {
  @Prop() street!: string;
  @Prop() city!: string;
}

@Schema({ timestamps: true })
class Student {
  @Prop({ required: true }) name!: string;
  @Prop({ type: Address }) address!: Address;   // embedded subdocument
}
```

Resulting document (one round-trip reads everything):

```
{
  _id: "65f...",
  name: "Rahul",
  address: { street: "MG Road", city: "Pune" }
}
```

Use embedding when:

- the related data is only ever read/used together with the parent (you never query addresses on their own)
- the child doesn't need to be shared across multiple parents
- the child list per parent is small and bounded (a few addresses, not thousands of orders)

This is the Mongo-native default — favoring one document per read is WHY Mongo exists as an alternative to Postgres's normalized tables (theory/37 section 7).

2. REFERENCING

Store an ObjectId instead of the actual data — same idea as course_id INT REFERENCES courses(id) in theory/37 section 7, just without the database enforcing it.

```
// course.schema.ts
@Schema({ timestamps: true })
export class Course {
  @Prop({ required: true }) title!: string;
  @Prop({ required: true }) credits!: number;
}
export const CourseSchema = SchemaFactory.createForClass(Course);

// student.schema.ts
import { Types } from 'mongoose';

@Schema({ timestamps: true })
export class Student {
  @Prop({ required: true }) name!: string;
  @Prop({ required: true }) age!: number;

  @Prop({ type: Types.ObjectId, ref: 'Course' })
  course!: Types.ObjectId;   // just an id, not the actual course data
}
```

`ref: 'Course'` doesn't touch the database at all — it's metadata Mongoose reads ONLY when you call .populate('course') later. Without that call, `student.course` is just a raw ObjectId string, same as reading course_id off a SQL row without doing the JOIN.

Use referencing when:

- the related data has its own independent lifecycle (a Course exists whether or not any Student is enrolled in it)
- many documents point at the same related doc (100 students → 1 Course — embedding would duplicate that Course 100 times)
- the child collection is large/unbounded (a Student's orders over years) — embedding would make the parent document grow forever, and Mongo caps a single document at 16MB

3. populate() — THE JOIN-EQUIVALENT FOR REFERENCES

```
async findOne(id: string): Promise<StudentDocument> {
  const student = await this.studentModel
    .findById(id)
    .populate('course')   // replaces the ObjectId with the full Course doc
    .exec();
  ...
}
```

Without .populate('course'):

```
{ _id: "...", name: "Rahul", course: "65f0a1..." }   // just an id
```

With .populate('course'):

```
{ _id: "...", name: "Rahul", course: { _id: "65f0a1...", title: "CS", credits: 4 } }
```

Under the hood populate() is Mongoose running a SECOND query (effectively `Course.findById(...)`) and stitching the result in — it is NOT a real database-level JOIN like SQL's, which is a single operation. This is the same "N round-trips instead of 1" cost every ODM/ORM pays for relational-style reads on a document database — worth knowing when you're populating a list of 1000 students each with a course (populate batches this into one extra query per relation, not per document, but it's still a second query, not a free join).

Selecting only some fields from the populated doc (like SQL's `SELECT students.name, courses.title`):

```
.populate('course', 'title credits')   // omits _id extras, timestamps etc.
```

Populating a relation on findAll() too:

```
async findAll(): Promise<StudentDocument[]> {
  return this.studentModel.find().populate('course').exec();
}
```

4. ONE-TO-MANY THE OTHER DIRECTION — ARRAY OF REFERENCES

A Course with many enrolled Students (the reverse of the relation above) stores an ARRAY of ObjectIds instead of one:

```
@Schema({ timestamps: true })
export class Course {
  @Prop({ required: true }) title!: string;

  @Prop({ type: [Types.ObjectId], ref: 'Student' })
  students!: Types.ObjectId[];
}

await this.courseModel.findById(id).populate('students').exec();
// -> { title: "CS", students: [ {full Student doc}, {full Student doc}, ... ] }
```

Same ref mechanism as section 2, just an array of ids instead of one — Mongoose populates every element in a single extra query.

5. MANY-TO-MANY

No join table needed the way SQL requires one (theory/37 section 7 covers SQL's version, which needs an intermediate table like enrollments(student_id, course_id)). Mongo's usual pattern: an array of references on EACH side.

```
// student.schema.ts
@Prop({ type: [Types.ObjectId], ref: 'Course' })
courses!: Types.ObjectId[];

// course.schema.ts
```

```
@Prop({ type: [Types.ObjectId], ref: 'Student' })
students!: Types.ObjectId[];
```

Tradeoff: you now maintain BOTH sides yourself whenever the relationship changes (push the course's id onto the student's array AND push the student's id onto the course's array) — Mongo won't keep them in sync for you the way a single SQL junction-table row would. If the relationship itself needs its own data (e.g. an enrollment date, a grade), model it as its own collection instead — same idea as

SQL's join table, just still without enforced foreign keys:

```
@Schema({ timestamps: true })
export class Enrollment {
  @Prop({ type: Types.ObjectId, ref: 'Student', required: true })
  student!: Types.ObjectId;

  @Prop({ type: Types.ObjectId, ref: 'Course', required: true })
  course!: Types.ObjectId;

  @Prop() grade!: string;
}
```

6. WHAT MONGO WON'T DO FOR YOU (vs SQL's FOREIGN KEY)

`ref: 'Course'` is a hint for populate() — it is NOT a constraint. Nothing stops you from saving a Student with a `course` field pointing at an ObjectId that doesn't exist in the courses collection, and nothing stops you from deleting a Course while Students still reference it. Contrast with theory/37 section 10, where `course_id INT REFERENCES courses(id)` makes Postgres itself reject the bad insert.

If you need that guarantee in Mongo, you enforce it in application code (check the referenced doc exists before saving) or reach for transactions (theory/37 section 12 — Mongo has an equivalent, but it's opt-in and heavier than SQL's default behavior).

★ RULE OF THUMB

Embed when the child is small, bounded, and always read together with the parent. Reference (ObjectId + ref) when the child has its own identity, is shared across parents, or can grow without bound. populate() is how you turn a stored reference back into real data at read time — it's a second query Mongoose runs for you, not a database JOIN. Mongo enforces none of this at the database level the way SQL's FOREIGN KEY does — referential integrity for references is on your application code.

See also: theory/31-mongoose-schema-decorators.txt, theory/32-injecting-mongoose-models.txt, theory/30-mongodb-connection.txt, theory/37-sql-cheatsheet.txt (SQL JOIN/foreign-key comparison)